UNIT-4
Collections in Java

The Java collections framework provides a set of interfaces and classes to implement various data structures and algorithms.

Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.

The Java collections classes and interfaces are defined inside java.util package.

Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

What is a Java Collection Framework?

A Java collection framework provides an architecture to store and manipulate a group of objects. A Java collection framework includes the following:

- Interfaces
- Classes
- Algorithm

Interfaces: Interface in Java refers to the abstract data types. They allow Java collections to be manipulated independently from the details of their representation. Also, they form a hierarchy in object-oriented programming languages.

Classes: Classes in Java are the implementation of the collection interface. It basically refers to the data structures that are used again and again.

Algorithm: Algorithm refers to the methods which are used to perform operations such as searching and sorting, on objects that implement collection interfaces. Algorithms are polymorphic in nature as the same method can be used to take many forms or you can say perform different implementations of the Java collection interface.

Why use Java collection?

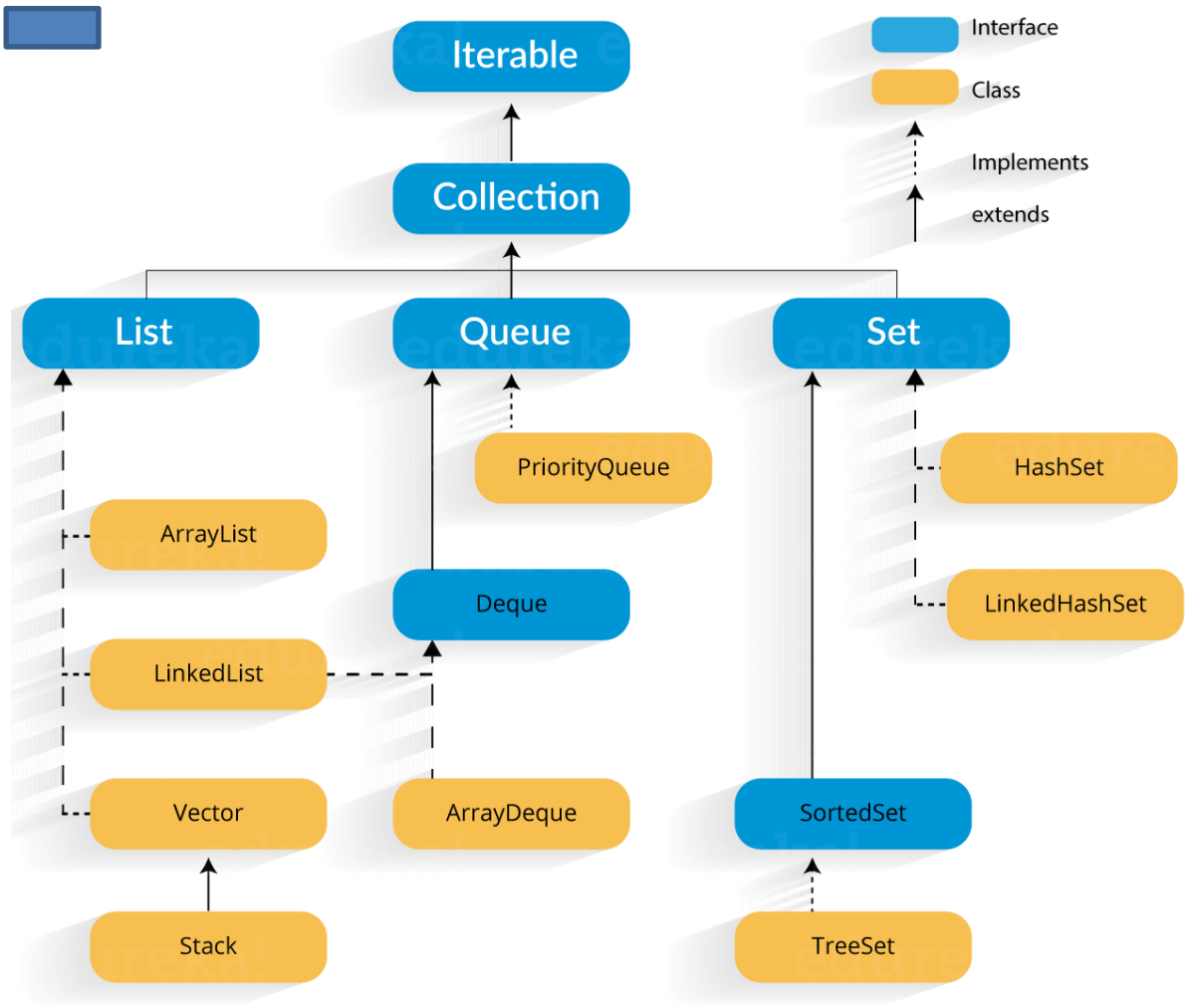
There are several benefits of using Java collections such as:

- Reducing the effort required to write the code by providing useful data structures and algorithms
- Java collections provide high-performance and high-quality data structures and algorithms thereby increasing the speed and quality

- Unrelated APIs can pass collection interfaces back and forth
- Decreases extra effort required to learn, use, and design new API's
- Supports reusability of standard data structures and algorithms

Java Collection Framework Hierarchy

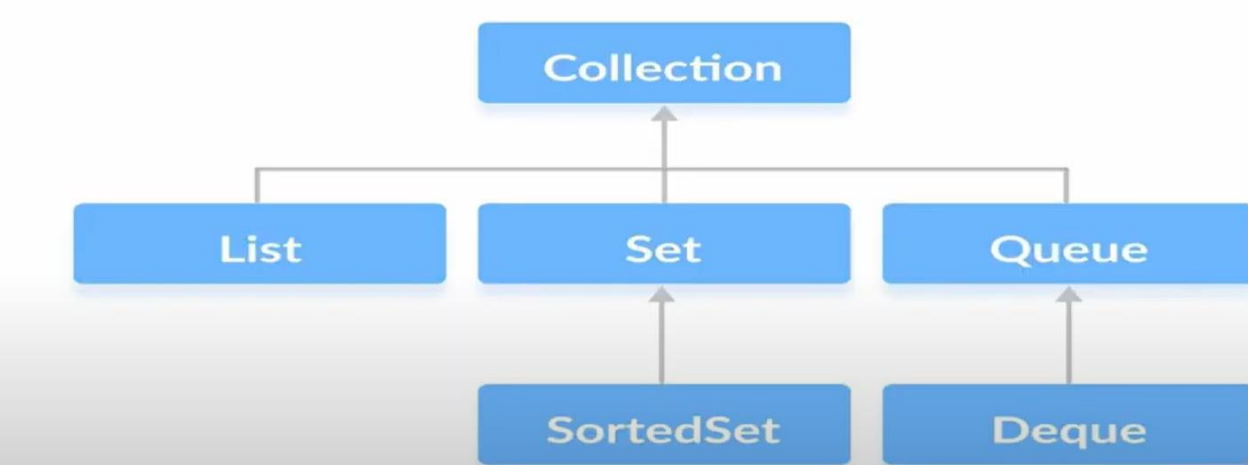
As we have learned Java collection framework includes interfaces and classes. Now, let us see the Java collections framework hierarchy.

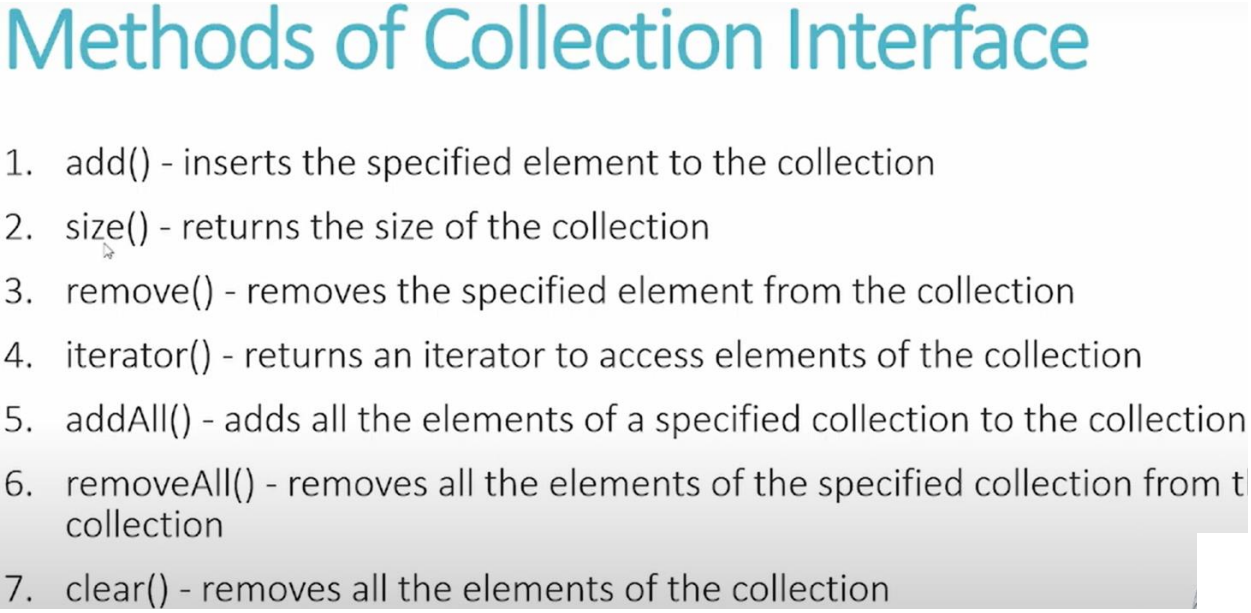


Java Collections : Interface

Iterator interface : Iterator is an interface that iterates the elements. It is used to traverse the list and modify the elements. Iterator interface has three methods which are mentioned below:

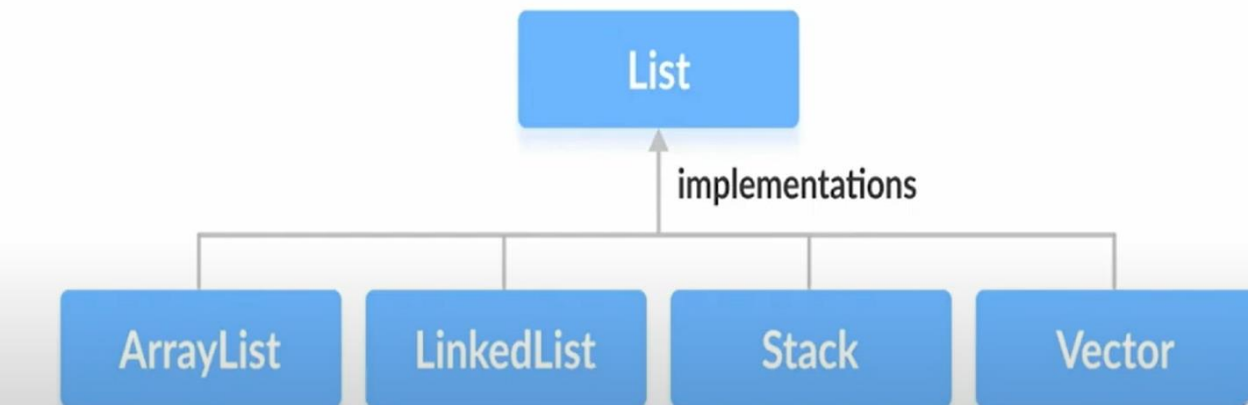
1. **public boolean hasNext()** – This method returns true if the iterator has more elements.
2. **public object next()** – It returns the element and moves the cursor pointer to the next element.
3. **public void remove()** – This method removes the last elements returned by the iterator.





Java collections: List

A List is an ordered Collection of elements which may contain duplicates. It is an interface that extends the Collection interface. Lists are further classified into the following:

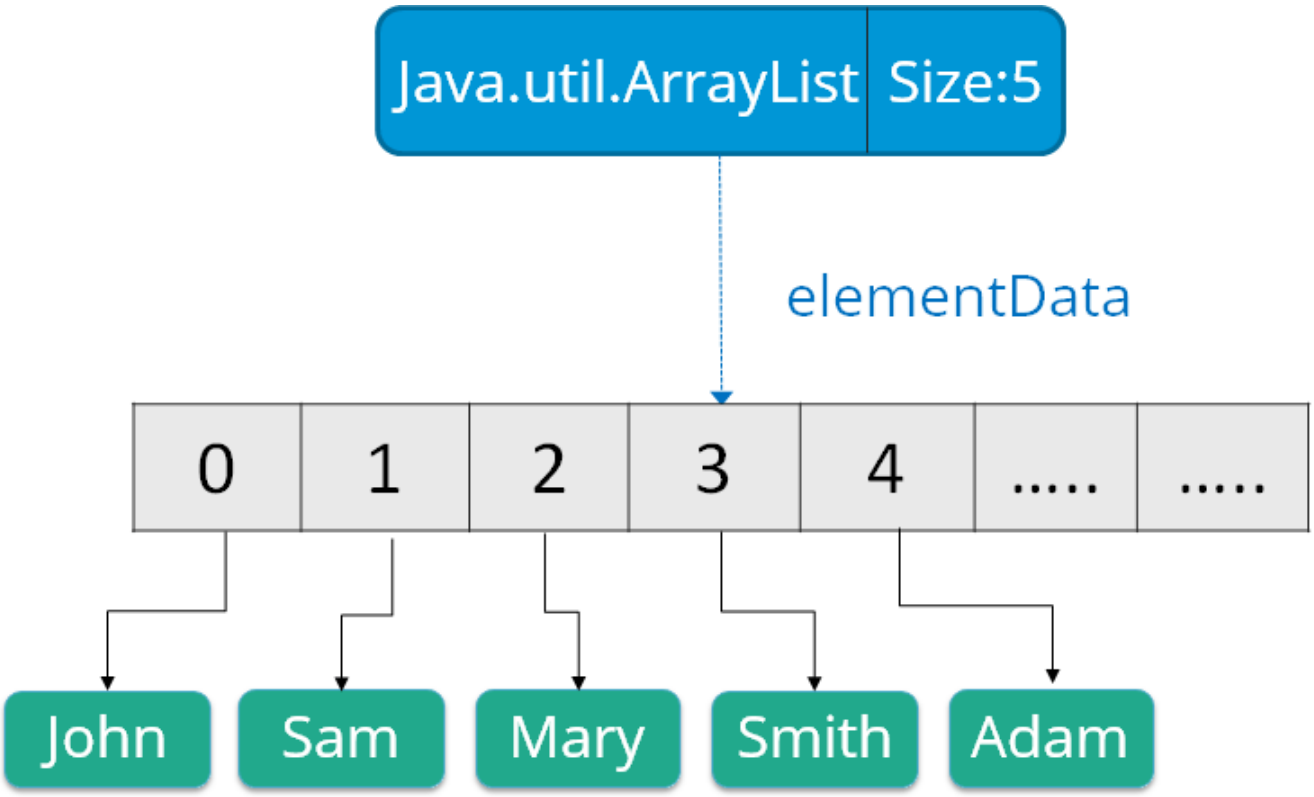


Methods of List Interface

1. set() - changes elements of lists
2. get() - helps to randomly access elements from lists
3. toArray() - converts a list into an array
4. contains() - returns true if a list contains specified element

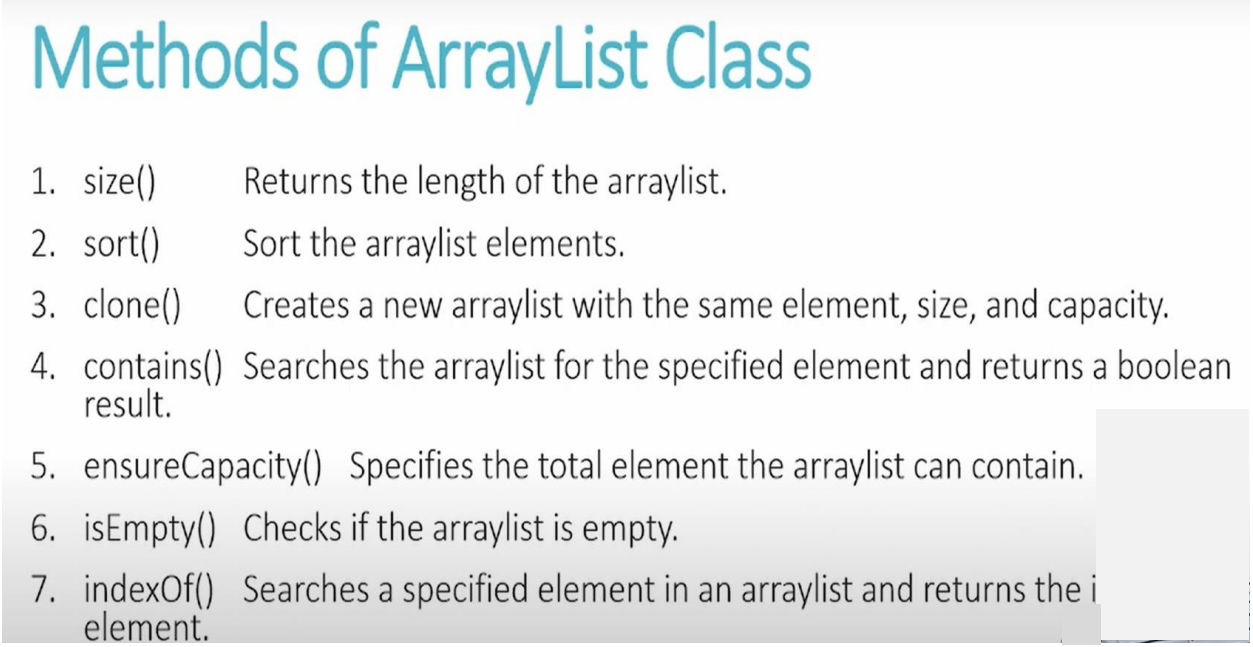
1. Array list:

ArrayList is the implementation of List Interface where the elements can be dynamically added or removed from the list. Also, the size of the list is increased dynamically if the elements are added more than the initial size.



Syntax:

ArrayList object = new ArrayList ();



Array list with a programmatic example:

```
import java.util.*;
class ArrayListExample{
public static void main(String args[]){

ArrayList al=new ArrayList();  // creating array list
al.add("Ram");                 // adding elements
al.add("Rahul");
Iterator itr=al.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

Output

```
Ram
Rahul
```

Iterator in Java Collection

In Java, an Iterator is one of the Java cursors.

Java Iterator is an interface that is practiced in order to iterate over a collection of Java object components entirety one by one.

Advantages of Java Iterator

1. The user can apply these iterators to any of the classes of the Collection framework.
2. In Java Iterator, we can use both of the read and remove operations.
3. If a user is working with a for loop, they cannot modernize(add/remove) the Collection, whereas, if they use the Java Iterator, they can simply update the Collection.

Disadvantages of Java Iterator

The Java Iterator only preserves the iteration in the forward direction. In simple words, the Java Iterator is a uni-directional Iterator.

Java Iterator Methods

1. hasNext()
2. next()
3. remove()

How to create iterator
Syntax:
Iterator <ClassName>object=collectionclass.iterator();

2.vector

Vectors : Vectors are similar to arrays, where the elements of the vector object can be accessed via an index into the vector. Vector implements a dynamic array. Also, the vector is not limited to a specific size, it can shrink or grow automatically whenever required. It is similar to ArrayList, but with two differences :

- Vector is synchronized.
- Vector contains many legacy methods that are not part of the collections framework.

Java Vector vs. ArrayList
The Vector class synchronizes each individual operation. This means whenever we want to perform some operation on vectors, the Vector class automatically applies a lock to that operation.

Creating a Vector

```
Vector<Type> vector = new Vector<>();
```

Methods

1. add(element) - adds an element to vectors
2. get(index) - returns an element specified by the index
3. iterator() - returns an iterator object to sequentially access vector elements
4. remove(index) - removes an element from specified position
5. removeAll() - removes all the elements
6. clear() - removes all elements. It is more efficient than removeAll()
7. set() - changes an element of the vector
8. size() - returns the size of the vector
9. toArray() - converts the vector into an array

Example:

```
import java.io.*;
import java.util.*;
class vector1
{
   public static void main(String[] args)
   {
      int n = 5;
      Vector<Integer> v = new Vector<Integer>(n);
      for (int i = 1; i <= n; i++)
         v.add(i);
      System.out.println(v);
      v.remove(3);
      System.out.println(v);
      for (int i = 0; i < v.size(); i++)

         // Printing elements one by one
         System.out.print(v.get(i) + " ");
   }
}
```

Output

```
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
1 2 3 5
```

Note:

- *If the vector increment is not specified then it's capacity will be doubled in every increment cycle.*
- *The capacity of a vector cannot be below the size, it may equal to it.*

Most Frequently used methods of collections class

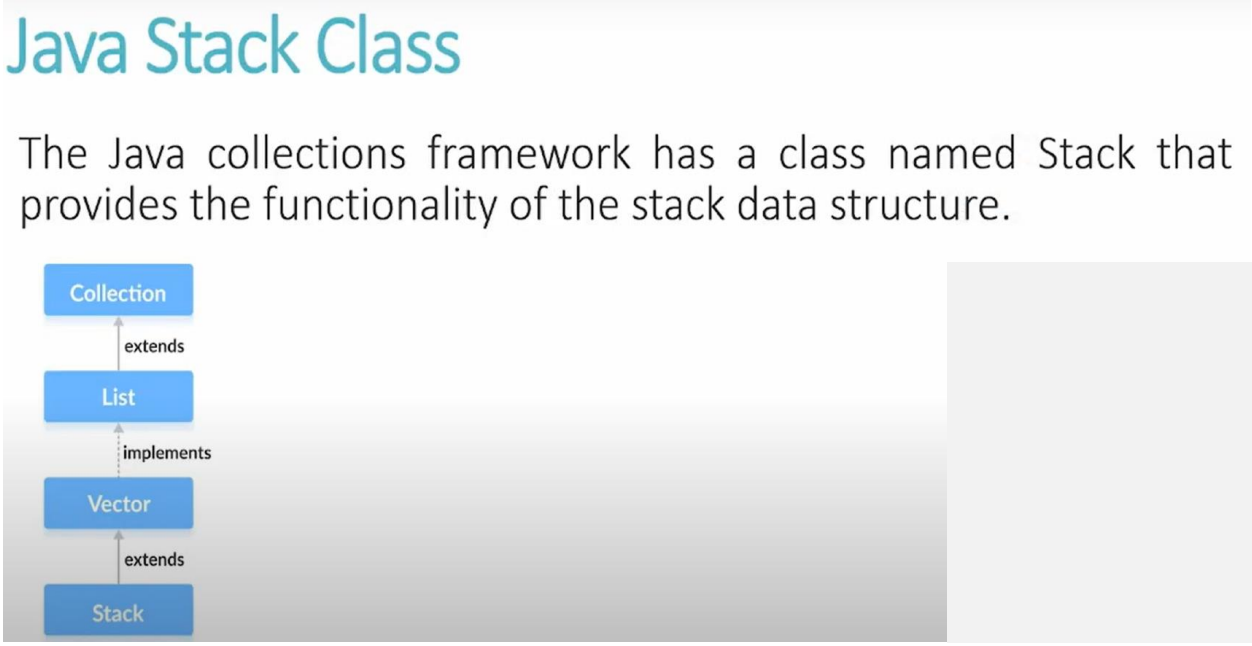
Collections class declaration

Let's see the declaration for java.util.Collections class.

1. **public class** Collections **extends** Object

SN	Modifier & Type	Methods	Descriptions
1)	static <T> boolean	addAll()	It is used to adds all of the specified elements to the specified collection.
2)	static <T> int	binarySearch()	It searches the list for the specified object and returns their position in a sorted list.
3)	static <T> void	copy()	It is used to copy all the elements from one list into another list.
4)	static <T extends Object & Comparable<? super T>> T	max()	It is used to get the maximum value of the given collection, according to the natural ordering of its elements.
5)	static <T extends Object & Comparable<? super T>> T	min()	It is used to get the minimum value of the given collection, according to the natural ordering of its elements.

6)	static <T> boolean	replaceAll()	It is used to replace all occurrences of one specified value in a list with the other specified value.
7)	static void	reverse()	It is used to reverse the order of the elements in the specified list.
8)	static <T extends Comparable<? super T>>void	sort()	It is used to sort the elements presents in the specified list of collection in ascending order.
9)	static void	swap()	It is used to swap the elements at the specified positions in the specified list.

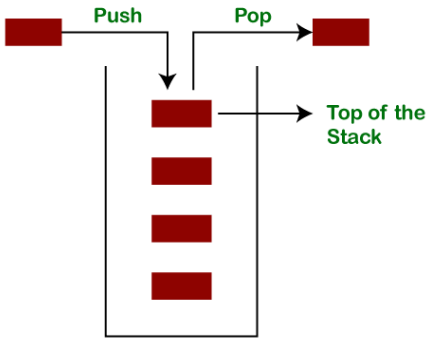


2. Stack Classes

The **stack** is a linear data structure that is used to store the collection of objects. It is based on **Last-In-First-Out** (LIFO). Java collection framework provides many interfaces and classes to store the collection of objects. One of them is the **Stack class** that provides different operations such as push, pop, search, etc.

In this section, we will discuss the **Java Stack class**, its **methods,** and **implement** the stack data structure in a Java program. But before moving to the Java Stack class have a quick view of how the stack works.

The stack data structure has the two most important operations that are **push** and **pop**. The push operation inserts an element into the stack and pop operation removes an element from the top of the stack. Let's see how they work on stack.



Creating a Stack

Stack<type> stk = new Stack<>();

Methods of the Stack Class

Method	Modifier and Type	Method Description
empty()	boolean	The method checks the stack is empty or not.
push(E item)	E	The method pushes (insert) an element onto the top of the stack.
pop()	E	The method removes an element from the top of the stack and returns the same element as the value of that function.
peek()	E	The method looks at the top element of the stack without removing it.
search(Object o)	int	The method searches the specified object and returns the position of the object.

Example:

```
import java.util.Stack;
public class StackEmptyMethodExample
{
```

```
public static void main(String[] args)
{
//creating an instance of Stack class
Stack<Integer> stk= new Stack<>();
// checking stack is empty or not
boolean result = stk.empty();
System.out.println("Is the stack empty? " + result);
// pushing elements into stack
stk.push(78);
stk.push(113);
stk.push(90);
stk.push(120);
//prints elements of the stack
System.out.println("Elements in Stack: " + stk);
result = stk.empty();
System.out.println("Is the stack empty? " + result);
}
}
```

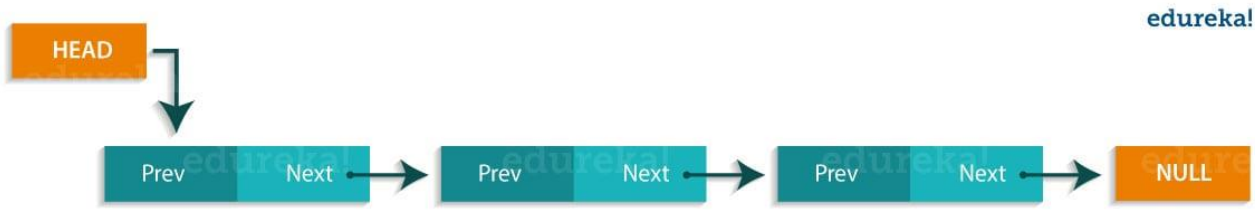
4. Linked List:

Linked List: Linked List is a sequence of links which contains items. Each link contains a connection to another link.

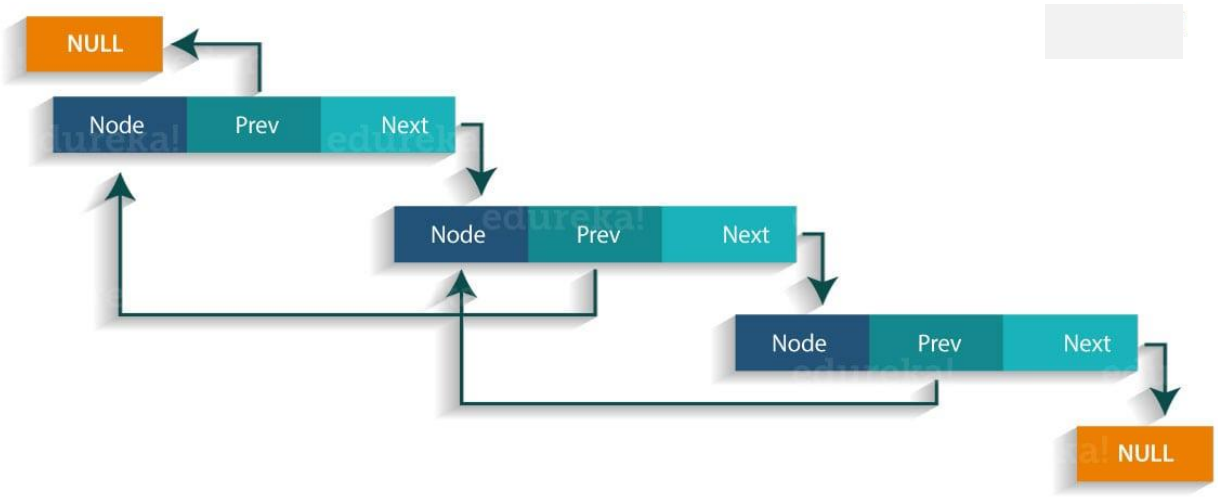
Java Linked List class uses two types of Linked list to store the elements:

- Singly Linked List
- Doubly Linked List

Singly Linked List: In a singly Linked list each node in this list stores the data of the node and a pointer or reference to the next node in the list. Refer to the below image to get a better understanding of single Linked list.



Doubly Linked List: In a doubly Linked list, it has two references, one to the next node and another to previous node. You can refer to the below image to get a better understanding of doubly linked list.



Some of the methods in the linked list are listed below:

Method	Description
boolean add(Object o)	It is used to append the specified element to the end of the vector.
boolean contains(Object o)	Returns true if this list contains the specified element.
void add (int index, Object element)	Inserts the element at the specified element in the vector.
void addFirst(Object o)	It is used to insert the given element at the beginning.
void addLast(Object o)	It is used to append the given element to the end.
int size()	It is used to return the number of elements in a list
boolean remove(Object o)	Removes the first occurrence of the specified element from this list.
int indexOf(Object element)	Returns the index of the first occurrence of the specified element in this list, or -1.

int lastIndexOf(Object element)	Returns the index of the last occurrence of the specified element in this list, or -1.

linked list with a programmatic example:

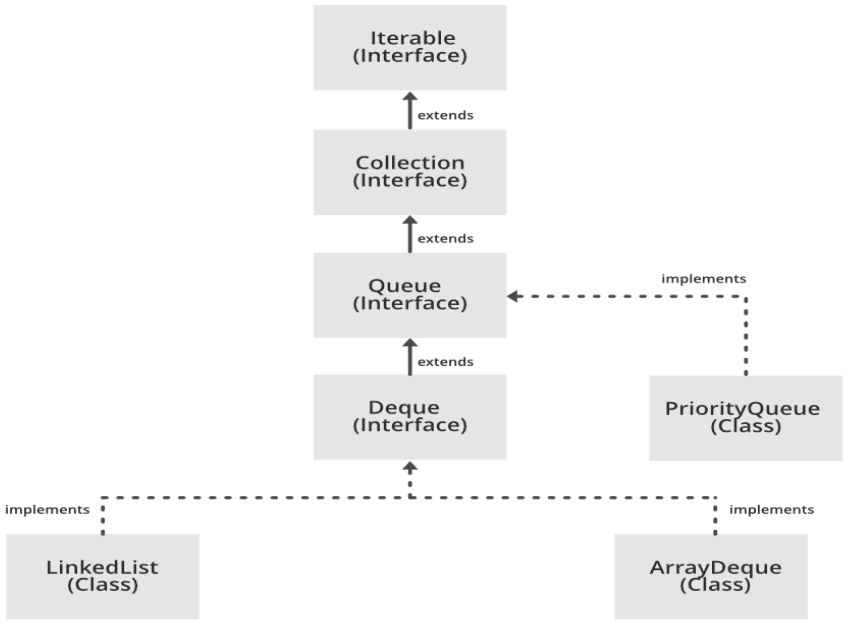
```
import java.util.*;
public class LinkedlistExample{
    public static void main(String args[]){
        LinkedList<String> al=new LinkedList<String>();// creating linked
list
        al.add("Rachit"); // adding elements
        al.add("Rahul");
        al.add("Rajat");
        Iterator<String> itr = al.iterator();
        while(itr.hasNext()){
            System.out.println(itr.next());
        }
    }
}
```

The output of the above program would be:

```
Rachit
Rahul
Rajat
```

Queue Interface In Java

The Queue interface is present in java.util package and extends the Collection interface is used to hold the elements about to be processed in FIFO(First In First Out) order. It is an ordered list of objects with its use limited to inserting elements at the end of the list and deleting elements from the start of the list, (i.e.), it follows the **FIFO** or the First-In-First-Out principle.



Creating Queue Objects:

```
Queue<Obj> queue = new PriorityQueue<Obj> ();
```

The Queue interface provides several methods for adding, removing, and inspecting elements in the queue. Here are some of the most commonly used methods:

add(element): Adds an element to the rear of the queue. If the queue is full, it throws an exception.
offer(element): Adds an element to the rear of the queue. If the queue is full, it returns false.
remove(): Removes and returns the element at the front of the queue. If the queue is empty, it throws an exception.
poll(): Removes and returns the element at the front of the queue. If the queue is empty, it returns null.

element(): Returns the element at the front of the queue without removing it. If the queue is empty, it throws an exception.

peek(): Returns the element at the front of the queue without removing it. If the queue is empty, it returns null.

Example:

```
import java.util.LinkedList;
import java.util.Queue;
public class Queue1
 {
        public static void main(String[] args)
 {
                Queue<String> queue = new LinkedList<>();
                queue.add("apple");
                queue.add("banana");
                queue.add("cherry");
                System.out.println("Queue: " + queue);

                String front = queue.remove();
                System.out.println("Removed element: " + front);

                // print the updated queue
                System.out.println("Queue after removal: " + queue);

                // add another element to the queue
                queue.add("date");

                // peek at the element at the front of the queue
                String peeked = queue.peek();
                System.out.println("Peeked element: " + peeked);
                // print the updated queue
                System.out.println("Queue after peek: " + queue);
        }}
```

Output

```
Queue: [apple, banana, cherry]
Removed element: apple
Queue after removal: [banana, cherry]
Peeked element: banana
Queue after peek: [banana, cherry, date]
```

Example2:

```
import java.util.LinkedList;
import java.util.Queue;
public class Queue2{
        public static void main(String[] args)
        {
        Queue<Integer> q= new LinkedList<>();
                for (int i = 0; i < 5; i++)
                        q.add(i);
        System.out.println("Elements of queue "+ q);
                // To remove the head of queue.
        int removedele = q.remove();
        System.out.println("removed element-"+ removedele);
       System.out.println(q);
  // To view the head of queue
        int head = q.peek();
                System.out.println("head of queue-"+ head);
                int size = q.size();
                System.out.println("Size of queue-"+ size);
        }
}
```

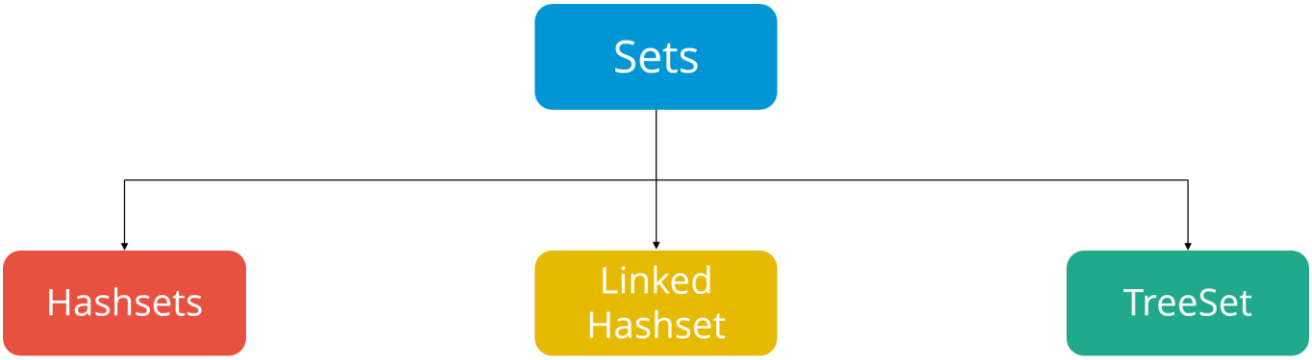
Output:

Output

```
Elements of queue [0, 1, 2, 3, 4]

removed element-0

[1, 2, 3, 4]

head of queue-1

Size of queue-4
```

Java Collections: Sets

A Set refers to a collection that cannot contain duplicate elements. It is mainly used to model the mathematical set abstraction. Set has its implementation in various classes such as HashSet, TreeSetand LinkedHashSet.



The **set** is an interface available in the **java.util** package. The **set** interface extends the Collection interface. An unordered collection or list in which duplicates are not allowed is referred to as a **collection interface**. The set interface is used to create the mathematical set. The set interface use collection interface's methods to avoid the insertion of the same elements. **SortedSet** and **NavigableSet** are two interfaces that extend the set implementation.

creating LinkedHashSet using the Set

```
Set<String> data = new LinkedHashSet<String>();
```

SetExample1.

```
import java.util.*;
public class setExample{
   public static void main(String[] args)
   {
      Set<String> data = new LinkedHashSet<String>();
      data.add("Raju");
      data.add("Stuti");
      data.add("Pankaj");
      data.add("Sumit");

      System.out.println(data);
   }
}
```

Output:

[Raju, Stuti, Pankaj, Sumit]

Operations on the Set Interface

- **Intersection:** The intersection operation returns all those elements which are present in both the set. The intersection of set1 and set2 will be [33, 45, 55].
- **Union:** The union operation returns all the elements of set1 and set2 in a single set, and that set can either be set1 or set2. The union of set1 and set2 will be [2, 3, 12, 22, 33, 34, 45, 55, 66, 77, 83].
- **Difference:** The difference operation deletes the values from the set which are present in another set. The difference of the set1 and set2 will be [66, 34, 22, 77].

In set, **addAll()** method is used to perform the union, **retainAll()** method is used to perform the intersection and **removeAll()** method is used to perform difference. Let's take an example to understand how these methods are used to perform the intersection, union, and difference operations.

SetExample2(example of union, intersection and difference)

```
import java.util.*;
public class SetOperations
{
   public static void main(String args[])
   {
      Integer[] A = {22, 45,33, 66, 55, 34, 77};
      Integer[] B = {33, 2, 83, 45, 3, 12, 55};

      Set<Integer> set1 = new HashSet<Integer>();
      set1.addAll(Arrays.asList(A));
      Set<Integer> set2 = new HashSet<Integer>();
      set2.addAll(Arrays.asList(B));

      // Finding Union of set1 and set2
      Set<Integer> union_data = new HashSet<Integer>(set1);
      union_data.addAll(set2);
      System.out.print("Union of set1 and set2 is:");
      System.out.println(union_data);

      // Finding Intersection of set1 and set2
      Set<Integer> intersection_data = new HashSet<Integer>(set1);
      intersection_data.retainAll(set2);
      System.out.print("Intersection of set1 and set2 is:");
```

```
        System.out.println(intersection_data);

        // Finding Difference of set1 and set2
        Set<Integer> difference_data = new HashSet<Integer>(set1);
        difference_data.removeAll(set2);
        System.out.print("Difference of set1 and set2 is:");
        System.out.println(difference_data);
    }
}
```

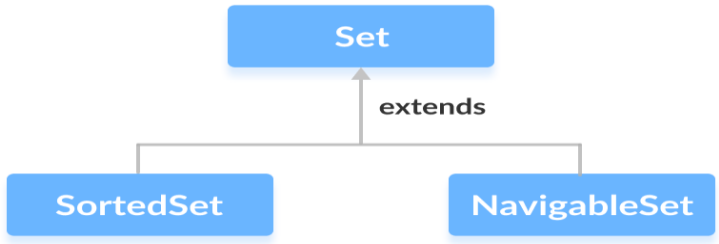
Output:

```
C:\Users\ajeet\OneDrive\Desktop\programs>java setOperations
Union of set1 and set2 is:[33, 66, 34, 2, 83, 3, 22, 55, 12, 45, 77]
Intersection of set1 and set2 is:[33, 55, 45]
Difference of set1 and set2 is:[66, 34, 22, 77]
```

Interfaces that extend Set

The `Set` interface is also extended by these subinterfaces:

1. SortedSet
2. NavigableSet



How to use Set?

// Set implementation using HashSet
Set<String> animals = new HashSet<>();

Methods of Set

The `Set` interface includes all the methods of the `Collection` interface. It's because `Collection` is a super interface of `Set`.
Some of the commonly used methods of the `Collection` interface that's also available in the `Set` interface are:

- **add()** - adds the specified element to the set
- **addAll()** - adds all the elements of the specified collection to the set

- **iterator()** - returns an iterator that can be used to access elements of the set sequentially
- **remove()** - removes the specified element from the set
- **removeAll()** - removes all the elements from the set that is present in another specified set
- **retainAll()** - retains all the elements in the set that are also present in another specified set
- **clear()** - removes all the elements from the set
- **size()** - returns the length (number of elements) of the set
- **toArray()** - returns an array containing all the elements of the set
- **contains()** - returns `true` if the set contains the specified element
- **containsAll()** - returns `true` if the set contains all the elements of the specified collection
- **hashCode()** - returns a hash code value (address of the element in the set)

Implementation of the Set Interface

1. Implementing HashSet Class

```
import java.util.Set;
import java.util.HashSet;
class Main {
   public static void main(String[] args) {
      // Creating a set using the HashSet class
      Set<Integer> set1 = new HashSet<>();
      // Add elements to the set1
      set1.add(2);
      set1.add(3);
      System.out.println("Set1: " + set1);

      // Creating another set using the HashSet class
      Set<Integer> set2 = new HashSet<>();
      // Add elements
      set2.add(1);
      set2.add(2);
      System.out.println("Set2: " + set2);
      // Union of two sets
      set2.addAll(set1);
      System.out.println("Union is: " + set2);
   }
}
```

Output

```
Set1: [2, 3]
Set2: [1, 2]
Union is: [1, 2, 3]
```

2. Implementing TreeSet Class

```
import java.util.Set;
import java.util.TreeSet;
import java.util.Iterator;
class Main {
   public static void main(String[] args) {
      // Creating a set using the TreeSet class
      Set<Integer> numbers = new TreeSet<>();
      // Add elements to the set
      numbers.add(2);
      numbers.add(3);
      numbers.add(1);
      System.out.println("Set using TreeSet: " + numbers);
      // Access Elements using iterator()
      System.out.print("Accessing elements using iterator(): ");
      Iterator<Integer> iterate = numbers.iterator();
      while(iterate.hasNext()) {
         System.out.print(iterate.next());
         System.out.print(", ");
      }}
}
```

Output

```
Set using TreeSet: [1, 2, 3]
Accessing elements using iterator(): 1, 2, 3,
```

Other Methods Of HashSet

Method	Description
`clone()`	Creates a copy of the `HashSet`
`contains()`	Searches the `HashSet` for the specified element and returns a boolean result
`isEmpty()`	Checks if the `HashSet` is empty

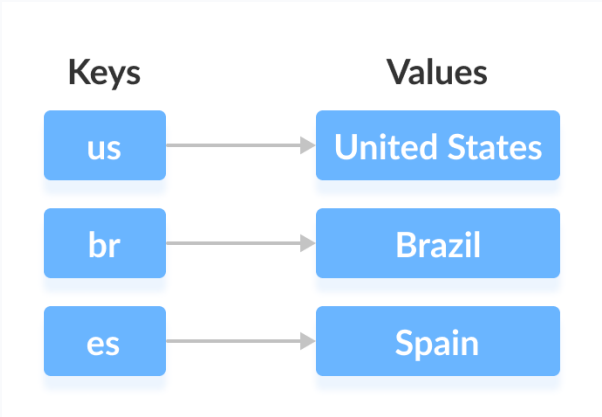
`size()`	Returns the size of the `HashSet`
`clear()`	Removes all the elements from the `HashSet`

Java Map Interface

The `Map` interface of the **Java collections framework** provides the functionality of the map data structure.

Working of Map

In Java, elements of `Map` are stored in **key/value** pairs. **Keys** are unique values associated with individual **Values**.
A map cannot contain duplicate keys. And, each key is associated with a single value.



We can access and modify values using the keys associated with them.

In the above diagram, we have values: `United States`, `Brazil`, and `Spain`. And we have corresponding keys: `us`, `br`, and `es`.

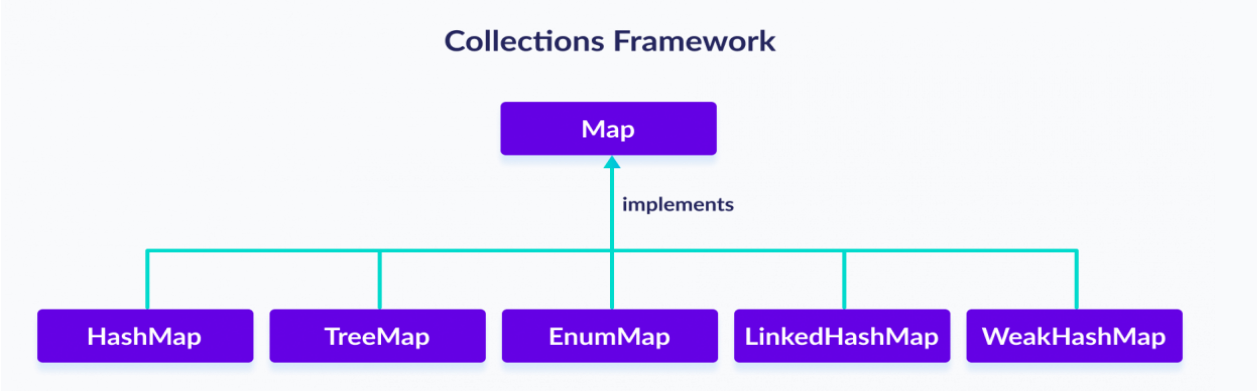
Note: The `Map` interface maintains 3 different sets:

- the set of keys
- the set of values
- the set of key/value associations (mapping).

Classes that implement Map

Since `Map` is an interface, we cannot create objects from it.

In order to use the functionalities of the `Map` interface, we can use these classes:



How to use Map?

In Java, we must import the `java.util.Map` package in order to use `Map`. Once we import the package, here's how we can create a map.

// Map implementation using HashMap
Map<Key, Value> numbers = new HashMap<>();

Methods of Map

The `Map` interface includes all the methods of the Collection interface. It is because `Collection` is a super interface of `Map`.
Besides methods available in the `Collection` interface, the `Map` interface also includes the following methods:

- **`put(K, V)`** - Inserts the association of a key `K` and a value `V` into the map. If the key is already present, the new value replaces the old value.
- **`putAll()`** - Inserts all the entries from the specified map to this map.
- **`putIfAbsent(K, V)`** - Inserts the association if the key `K` is not already associated with the value `V`.
- **`get(K)`** - Returns the value associated with the specified key `K`. If the key is not found, it returns `null`.
- **`getOrDefault(K, defaultValue)`** - Returns the value associated with the specified key `K`. If the key is not found, it returns the `defaultValue`.
- **`containsKey(K)`** - Checks if the specified key `K` is present in the map or not.
- **`containsValue(V)`** - Checks if the specified value `V` is present in the map or not.
- **`replace(K, V)`** - Replace the value of the key `K` with the new specified value `V`.
- **`replace(K, oldValue, newValue)`** - Replaces the value of the key `K` with the new value `newValue` only if the key `K` is associated with the value `oldValue`.
- **`remove(K)`** - Removes the entry from the map represented by the key `K`.
- **`remove(K, V)`** - Removes the entry from the map that has key `K` associated with value `V`.

- **keySet()** - Returns a set of all the keys present in a map.
- **values()** - Returns a set of all the values present in a map.
- **entrySet()** - Returns a set of all the key/value mapping present in a map.

Implementation of the Map Interface

1. Implementing HashMap Class

```
import java.util.Map;
import java.util.HashMap;
class Main {
   public static void main(String[] args) {
      // Creating a map using the HashMap
      Map<String, Integer> numbers = new HashMap<>();
      // Insert elements to the map
      numbers.put("One", 1);
      numbers.put("Two", 2);
      System.out.println("Map: " + numbers);
      // Access keys of the map
      System.out.println("Keys: " + numbers.keySet());
      // Access values of the map
      System.out.println("Values: " + numbers.values());
      // Access entries of the map
      System.out.println("Entries: " + numbers.entrySet());
      // Remove Elements from the map
      int value = numbers.remove("Two");
      System.out.println("Removed Value: " + value);
   }
}
```

Output

```
Map: {One=1, Two=2}
Keys: [One, Two]
Values: [1, 2]
Entries: [One=1, Two=2]
Removed Value: 2
```

2. Implementing TreeMap Class

```
import java.util.Map;
import java.util.TreeMap;
class Main {
   public static void main(String[] args) {
      // Creating Map using TreeMap
```

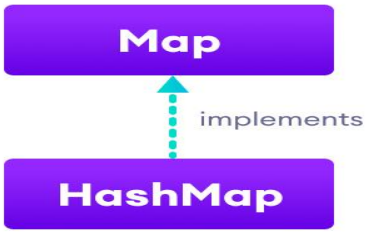
```
        Map<String, Integer> values = new TreeMap<>();
        // Insert elements to map
        values.put("Second", 2);
        values.put("First", 1);
        System.out.println("Map using TreeMap: " + values);

        // Replacing the values
        values.replace("First", 11);
        values.replace("Second", 22);
        System.out.println("New Map: " + values);

        // Remove elements from the map
        int removedValue = values.remove("First");
        System.out.println("Removed Value: " + removedValue);
    }
}
```

Output

```
Map using TreeMap: {First=1, Second=2}
New Map: {First=11, Second=22}
Removed Value: 11
```

Java HashMap

The `HashMap` class of the Java collections framework provides the functionality of the hash table data structure.
It stores elements in **key/value** pairs. Here, **keys** are unique identifiers used to associate each **value** on a map.
The `HashMap` class implements the Map interface.



Create a HashMap

In order to create a hash map, we must import the `java.util.HashMap` package

// hashMap creation with 8 capacity and 0.6 load factor

HashMap<K, V> numbers = new HashMap<>();

For example,

HashMap<String, Integer> numbers = new HashMap<>();

Here, the type of **keys** is `String` and the type of **values** is `Integer`.

Example 1: Create HashMap in Java

```
import java.util.HashMap;
class Main {
  public static void main(String[] args) {
    // create a hashmap
    HashMap<String, Integer> languages = new HashMap<>();
    // add elements to hashmap
    languages.put("Java", 8);
    languages.put("JavaScript", 1);
    languages.put("Python", 3);
    System.out.println("HashMap: " + languages);
  }
}
```

Output

```
HashMap: {Java=8, JavaScript=1, Python=3}
```

Basic Operations on Java HashMap

The `HashMap` class provides various methods to perform different operations on hashmaps. We will look at some commonly used arraylist operations in this tutorial:

- Add elements
- Access elements
- Change elements
- Remove elements

1. Add elements to a HashMap

```
import java.util.HashMap;

class Main {
  public static void main(String[] args) {
```

```
    // create a hashmap
    HashMap<String, Integer> numbers = new HashMap<>();

    System.out.println("Initial HashMap: " + numbers);
    // put() method to add elements
    numbers.put("One", 1);
    numbers.put("Two", 2);
    numbers.put("Three", 3);
    System.out.println("HashMap after put(): " + numbers);
  }
}
```

2. Access HashMap Elements

```
import java.util.HashMap;

class Main {
  public static void main(String[] args) {

    HashMap<Integer, String> languages = new HashMap<>();
    languages.put(1, "Java");
    languages.put(2, "Python");
    languages.put(3, "JavaScript");
    System.out.println("HashMap: " + languages);

    // get() method to get value
    String value = languages.get(1);
    System.out.println("Value at index 1: " + value);
  }
}
```

3. Change HashMap Value

```
import java.util.HashMap;

class Main {
  public static void main(String[] args) {

    HashMap<Integer, String> languages = new HashMap<>();
    languages.put(1, "Java");
    languages.put(2, "Python");
    languages.put(3, "JavaScript");
```

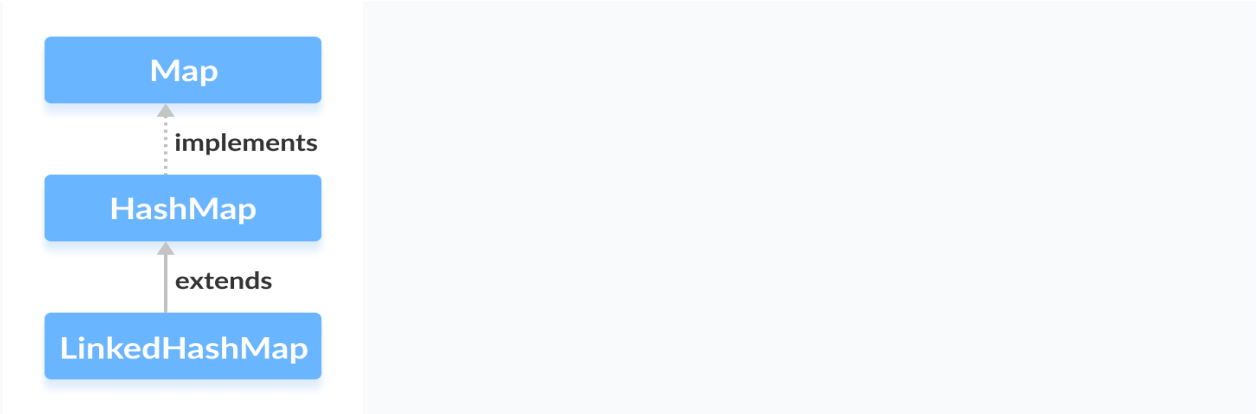
```
    System.out.println("Original HashMap: " + languages);

    // change element with key 2
    languages.replace(2, "C++");
    System.out.println("HashMap using replace(): " + languages);
  }
}
```

4. Remove HashMap Elements

```
import java.util.HashMap;

class Main {
  public static void main(String[] args) {

    HashMap<Integer, String> languages = new HashMap<>();
    languages.put(1, "Java");
    languages.put(2, "Python");
    languages.put(3, "JavaScript");
    System.out.println("HashMap: " + languages);

    // remove element associated with key 2
    String value = languages.remove(2);
    System.out.println("Removed value: " + value);

    System.out.println("Updated HashMap: " + languages);
  }
}
```

Java LinkedHashMap

The `LinkedHashMap` class of the Java collections framework provides the hash table and linked list implementation of the Map interface.
The `LinkedHashMap` interface extends the HashMap class to store its entries in a hash table. It internally maintains a doubly-linked list among all of its entries to order its entries.



Creating a LinkedHashMap

In order to create a linked hashmap, we must import the java.util.LinkedHashMap package first.

Methods of LinkedHashMap

The LinkedHashMap class provides methods that allow us to perform various operations on the map.

Insert Elements to LinkedHashMap

- `put()` - inserts the specified key/value mapping to the map
- `putAll()` - inserts all the entries from the specified map to this map
- `putIfAbsent()` - inserts the specified key/value mapping to the map if the specified key is not present in the map

Example:

```
import java.util.LinkedHashMap;
class Main {
    public static void main(String[] args) {
        // Creating LinkedHashMap of even numbers
        LinkedHashMap<String, Integer> evenNumbers = new LinkedHashMap<>();
        // Using put()
        evenNumbers.put("Two", 2);
        evenNumbers.put("Four", 4);
```

```
        System.out.println("Original LinkedHashMap: " + evenNumbers);
        // Using putIfAbsent()
        evenNumbers.putIfAbsent("Six", 6);
        System.out.println("Updated LinkedHashMap(): " + evenNumbers);

        //Creating LinkedHashMap of numbers
        LinkedHashMap<String, Integer> numbers = new LinkedHashMap<>();
        numbers.put("One", 1);

        // Using putAll()
        numbers.putAll(evenNumbers);
        System.out.println("New LinkedHashMap: " + numbers);
    }
}
```

Output

```
Original LinkedHashMap: {Two=2, Four=4}
Updated LinkedHashMap: {Two=2, Four=4, Six=6}
New LinkedHashMap: {One=1, Two=2, Four=4, Six=6}
```

Access LinkedHashMap Elements

1. Using entrySet(), keySet() and values()

entrySet() - returns a set of all the key/value mapping of the map
keySet() - returns a set of all the keys of the map
values() - returns a set of all the values of the map

```
import java.util.LinkedHashMap;
class Main {
    public static void main(String[] args) {
        LinkedHashMap<String, Integer> numbers = new LinkedHashMap<>();
        numbers.put("One", 1);
        numbers.put("Two", 2);
        numbers.put("Three", 3);
        System.out.println("LinkedHashMap: " + numbers);

        // Using entrySet()
        System.out.println("Key/Value mappings: " + numbers.entrySet());

        // Using keySet()
```

```
        System.out.println("Keys: " + numbers.keySet());

        // Using values()
        System.out.println("Values: " + numbers.values());
    }
}
```

Output

LinkedHashMap: {One=1, Two=2, Three=3}
Key/Value mappings: [One=1, Two=2, Three=3]
Keys: [One, Two, Three]
Values: [1, 2, 3]

Other Methods of LinkedHashMap

Method	Description
`clear()`	removes all the entries from the map
`containsKey()`	checks if the map contains the specified key and returns a boolean value
`containsValue()`	checks if the map contains the specified value and returns a boolean value
`size()`	returns the size of the map
`isEmpty()`	checks if the map is empty and returns a boolean value

LinkedHashMap Vs. HashMap

- Both the LinkedHashMap and the HashMap implements the Map interface. However, there exist some differences between them.
- LinkedHashMap maintains a doubly-linked list internally. Due to this, it maintains the insertion order of its elements.
- The LinkedHashMap class requires more storage than HashMap. This is because LinkedHashMap maintains linked lists internally.
- The performance of LinkedHashMap is slower than HashMap.

Java Hashtable class

Java Hashtable class implements a hashtable, which maps keys to values. It inherits Dictionary class and implements the Map interface.

Points to remember

- A Hashtable is an array of a list. Each list is known as a bucket. The position of the bucket is identified by calling the hashcode() method. A Hashtable contains values based on the key.
- Java Hashtable class contains unique elements.
- Java Hashtable class doesn't allow null key or value.
- Java Hashtable class is synchronized.
- The initial default capacity of Hashtable class is 11 whereas loadFactor is 0.75.

Hashtable class declaration

declaration for java.util.Hashtable class.

Java Hashtable Example

```
import java.util.*;
class Hashtable1{
 public static void main(String args[]){
  Hashtable<Integer,String> hm=new Hashtable<Integer,String>();

  hm.put(100,"Amit");
  hm.put(102,"Ravi");
  hm.put(101,"Vijay");
  hm.put(103,"Rahul");

  for(Map.Entry m:hm.entrySet()){
   System.out.println(m.getKey()+" "+m.getValue());
  }
 }
}
```

Difference between HashMap and Hashtable

HashMap	Hashtable
1) HashMap is **non synchronized**. It is not-thread safe and can't be shared between many threads without	Hashtable is **synchronized**. It is thread-safe and can be shared with many threads.

proper synchronization code.	
2) HashMap **allows one null key and multiple null values**.	Hashtable **doesn't allow any null key or value**.
3) HashMap is a **new class introduced in JDK 1.2**.	Hashtable is a **legacy class**.
4) HashMap is **fast**.	Hashtable is **slow**.
5) We can make the HashMap as synchronized by calling this code Map m = Collections.synchronizedMap(hashMap);	Hashtable is internally synchronized and can't be unsynchronized.
6) HashMap is **traversed by Iterator**.	Hashtable is **traversed by Enumerator and Iterator**.
7) Iterator in HashMap is **fail-fast**.	Enumerator in Hashtable is **not fail-fast**.
8) HashMap inherits **AbstractMap** class.	Hashtable inherits **Dictionary** class.

Sorting in Collection

We can sort the elements of:

1. String objects
2. Wrapper class objects
3. User-defined class objects

Collections class provides static methods for sorting the elements of a collection. If collection elements are of use TreeSet. However, we cannot sort the elements of List. Collections class provides methods for sorting the el elements.

Method of Collections class for sorting List elements

public void sort(List list): is used to sort the elements of List. List elements must be of the Comparable type.

Note: String class and Wrapper classes implement the Comparable interface. So if you store the objects of string or wrapper classes, it will be Comparable.

Example to sort string objects

```
import java.util.*;
class TestSort1{
public static void main(String args[]){

ArrayList<String> al=new ArrayList<String>();
al.add("Viru");
al.add("Saurav");
al.add("Mukesh");
al.add("Tahir");
Collections.sort(al);
Iterator itr=al.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
 }
}
}
```

Output:

```
Mukesh
Saurav
```

```
Tahir
Viru
```

Example to sort string objects in reverse order

```
import java.util.*;
class TestSort2{
public static void main(String args[]){

ArrayList<String> al=new ArrayList<String>();
       al.add("Viru");
        al.add("Saurav");
       al.add("Mukesh");
        al.add("Tahir");

        Collections.sort(al,Collections.reverseOrder());
       Iterator i=al.iterator();
        while(i.hasNext())
       {
           System.out.println(i.next());
       }
}
}
```

```
Viru
Tahir
Saurav
Mukesh
```

Example to sort Wrapper class objects

```
import java.util.*;
class TestSort3{
public static void main(String args[]){

ArrayList al=new ArrayList();
al.add(Integer.valueOf(201));
al.add(Integer.valueOf(101));
al.add(230);//internally will be converted into objects as Integer.valueOf(230)
Collections.sort(al);

Iterator itr=al.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
 }
}
```

```
}
```

Java Comparable interface

- Java Comparable interface is used to order the objects of the user-defined class. This interface is found in java.lang package and contains only one method named compareTo(Object).
- It provides a single sorting sequence only, i.e., you can sort the elements on the basis of single data member only. For example, it may be rollno, name, age or anything else.

compareTo(Object obj) method

public int compareTo(Object obj): It is used to compare the current object with the specified object. It returns

- positive integer, if the current object is greater than the specified object.
- negative integer, if the current object is less than the specified object.
- zero, if the current object is equal to the specified object.

Java Comparable Example

Example of the Comparable interface that sorts the list elements on the basis of age.

File: Student.java

```
class Student implements Comparable<Student>{
int rollno;
String name;
int age;
Student(int rollno,String name,int age){
this.rollno=rollno;
this.name=name;
this.age=age;
}
public int compareTo(Student st){
if(age==st.age)
return 0;
else if(age>st.age)
return 1;
else
return -1;
}
```

```
}
```

File: TestSort1.java

```
import java.util.*;
public class TestSort1{
public static void main(String args[]){
ArrayList<Student> al=new ArrayList<Student>();
al.add(new Student(101,"Vijay",23));
al.add(new Student(106,"Ajay",27));
al.add(new Student(105,"Jai",21));

Collections.sort(al);
for(Student st:al){
System.out.println(st.rollno+" "+st.name+" "+st.age);
}
}
}
```

```
105 Jai 21
101 Vijay 23
106 Ajay 27
```

Java Comparator interface

Java Comparator interface is used to order the objects of a user-defined class.

This interface is found in java.util package and contains 2 methods compare(Object obj1,Object obj2) and equals(Object element).

It provides multiple sorting sequences, i.e., you can sort the elements on the basis of any data member, for example, rollno, name, age or anything else.

Methods of Java Comparator Interface

Method	Description
public int compare(Object obj1, Object obj2)	It compares the first object with the second object.
public boolean equals(Object obj)	It is used to compare the current object with the specified object.
public boolean equals(Object obj)	It is used to compare the current object with the specified object.

Example:

File: Student.java

```
class Student {
  int rollno;
  String name;
 int age;
  Student(int rollno,String name,int age){
  this.rollno=rollno;
  this.name=name;
  this.age=age;
  }

  public int getRollno() {
     return rollno;
  }

  public void setRollno(int rollno) {
     this.rollno = rollno;
  }

  public String getName() {
     return name;
  }

  public void setName(String name) {
     this.name = name;
  }

  public int getAge() {
     return age;
  }

  public void setAge(int age) {
     this.age = age;
  }

  }
```

File: TestSort1.java

```
import java.util.*;
public class TestSort1{
public static void main(String args[]){
ArrayList<Student> al=new ArrayList<Student>();
al.add(new Student(101,"Vijay",23));
al.add(new Student(106,"Ajay",27));
al.add(new Student(105,"Jai",21));
/Sorting elements on the basis of name
Comparator<Student> cm1=Comparator.comparing(Student::getName);
Collections.sort(al,cm1);
System.out.println("Sorting by Name");
for(Student st: al){
System.out.println(st.rollno+" "+st.name+" "+st.age);
}
//Sorting elements on the basis of age
Comparator<Student> cm2=Comparator.comparing(Student::getAge);
Collections.sort(al,cm2);
System.out.println("Sorting by Age");
for(Student st: al){
System.out.println(st.rollno+" "+st.name+" "+st.age);
}
}
}
```

```
Sorting by Name
106 Ajay 27
105 Jai 21
101 Vijay 23
Sorting by Age
105 Jai 21
101 Vijay 23
106 Ajay 27
```

Properties class in Java

The **properties** object contains key and value pair both as a string. The java.util.Properties class is the subclass of Hashtable.

It can be used to get property value based on the property key. The Properties class provides methods to get data from the properties file and store data into the properties file. Moreover, it can be used to get the properties of a system.

An Advantage of the properties file

Recompilation is not required if the information is changed from a properties file: If any information is changed from the properties file, you don't need to recompile the java class. It is used to store information which is to be changed frequently.

Constructors of Properties class

Method	Description
Properties()	It creates an empty property list with no default values.
Properties(Properties defaults)	It creates an empty property list with the specified defaults.

Example of Properties class to get information from the properties file

To get information from the properties file, create the properties file first.

db.properties

```
user=system
password=oracle
```

Now, let's create the java class to read the data from the properties file.

Test.java

```
import java.util.*;
import java.io.*;
public class Test {
public static void main(String[] args)throws Exception{
    FileReader reader=new FileReader("db.properties");

    Properties p=new Properties();
    p.load(reader);

    System.out.println(p.getProperty("user"));
    System.out.println(p.getProperty("password"));
}
}
```

```
Output:system
       oracle
```

Example of Properties class to get all the system properties

By System.getProperties() method we can get all the properties of the system. Let's create the class that gets information from the system properties.

Test.java

```
import java.util.*;
import java.io.*;
public class Test {
public static void main(String[] args)throws Exception{

Properties p=System.getProperties();
Set set=p.entrySet();

Iterator itr=set.iterator();
while(itr.hasNext()){
Map.Entry entry=(Map.Entry)itr.next();
System.out.println(entry.getKey()+" = "+entry.getValue());
}

}
}
```